

Forms

This is important since **we must never mutate state directly in React!**

The Save

- When the user clicks the `<button type="submit">`, the `<form onSubmit={addNote}>` triggers.
 - `event.preventDefault()`: If you don't have this, the browser will act like it's 1999 and try to refresh the whole webpage to send the data to a server. This line says "Stop! Don't reload the page. React will handle this."
 - **Creating the Object**: You create `newObj`. Notice how you set content: `newNote`. You don't have to go searching for what the user typed; you already have it sitting safely in your state from the Hijack!
 - **Saving to State**: You use `setNotes([...notes, newObj])` to safely add the new box to the conveyor belt without mutating the original array.
 - **Clearing the Input**: `setNewNote("")` instantly wipes the text box clean so the user can type another note.
-

The Hijack (Controlled Component)

- Look at your input box: In normal HTML, the input box controls itself. But here, you have hijacked it.
 - When the user types the letter "H" on their keyboard, the `onChange` event instantly fires.
 - It triggers `handleNoteChange(event)`.
 - `event.target.value` is JavaScript's way of reaching into the HTML input box and grabbing the letter "H".
 - It calls `setNewNote('H')`.
 - React instantly re-renders the screen, and forces the input box to display the value of the state (`value={newNote}`).
-

Filtering Displayed Elements

- Let's add some new functionality to our application that allows us to only view the important notes.
 - just showed how to use Array Method called `Filter`.
-

Exercise Veridict (Summury)

Early Return "trick" to kill a function;

- (e.g)

```
const addPerson = (event) => {
  event.preventDefault();

  let isDuplicateName = persons.some((person) => person.name === newName);

  if (isDuplicateName) {
    alert(`${newName} is already added to phonebook`);
    return
  };

  const personObject = {
    name: newName
  }

  setPersons([...persons, personObject]);
  setNewName('')
};
```

- You didn't lack JavaScript concepts. You just hadn't seen the Early Return "trick" used to stop a function before. You knew what return does (you use it in every React component). You just didn't realize you were allowed to use a blank return; as a kill switch.
- if I did not put return after `alert` it will continuously goes down read remaining script; unfortunately app will get crash;
- Practice it into vanilla JS :-

```
filter(),
map()
reduce(),
some()
```